

A Dynamic Deep Reinforcement Learning-Bayesian Framework for Anomaly Detection

Jeremy Watts¹, Franco van Wyk, Shahrbanoo Rezaei², Yiyang Wang³, Neda Masoud⁴,
and Anahita Khojandi⁵, *Member, IEEE*

Abstract—To assure the successful operation of connected and automated vehicles, it is critical to detect and isolate anomalous and/or faulty information in a timely manner. To do so, anomaly detection techniques should be implemented in real-time where if the probability of anomalous information exceeds a certain threshold, the information is dealt with accordingly. Traditionally, the threshold for judging whether the data is anomalous is fixed and determined *a priori*. However, not only does this approach fail to account for the feedback obtained during a trip on the performance of the algorithms, but it also fails to respond to potential changes in rates of anomalies. Hence, it is important to develop an approach that can dynamically alter this threshold in response to exogenous factors to assure reliable and robust system operation. We develop a mathematical framework which utilizes a dynamic threshold for an anomaly classification algorithm in order to maximize the safety of a trip. Specifically, we develop and pair an anomaly classification algorithm based on convolutional neural networks (CNN), with a partially observable Markov decision process (POMDP) model. We solve the resulting POMDP model using the asynchronous advantage actor critic (A3C) deep reinforcement learning algorithm. The prescribed policy determines the anomaly classification threshold in real-time that maximizes the performance. Our numerical experiments show that the POMDP model outperforms state-of-the-art benchmarks, especially under more difficult to detect anomaly profiles.

Index Terms—Cyber-physical systems, connected and automated vehicles, anomaly detection, vehicle safety, deep reinforcement learning, dynamic model, partial information.

I. INTRODUCTION

IN RECENT times, the transportation community has directed considerable research efforts into the development of various aspects of an intelligent transportation system (ITS). An ITS includes numerous concepts such as traffic management, traveler information systems, and intelligent vehicle capabilities [1]. These measures aim to relieve traffic congestion, improve transportation mobility, reduce emissions, enhance user experience, and improve safety. Connected and automated vehicles (CAVs) are expected to be an integral part of achieving the aforementioned ITS objectives [2].

Manuscript received 12 July 2019; revised 18 August 2020, 6 January 2021, 7 June 2021, and 3 December 2021; accepted 13 August 2022. Date of publication 31 August 2022; date of current version 5 December 2022. The Associate Editor for this article was A. Amditis. (Corresponding author: Anahita Khojandi.)

Jeremy Watts, Shahrbanoo Rezaei, and Anahita Khojandi are with the Department of Industrial and Systems Engineering, The University of Tennessee Knoxville, Knoxville, TN 37996 USA (e-mail: khojandi@utk.edu).

Franco van Wyk is with the Department of Industrial Engineering, Stellenbosch University, Stellenbosch 7600, South Africa.

Yiyang Wang and Neda Masoud are with the Department of Civil and Environmental Engineering, University of Michigan, Ann Arbor, MI 48109 USA.

Digital Object Identifier 10.1109/TITS.2022.3200906

CAVs make use of an array of sensors and communication technologies to control vehicle operation in a safe and efficient manner. CAVs use sensors such as cameras, ultrasonic sensors, radar, and LiDAR to capture the driving environment in order to operate the vehicle autonomously (with varying degrees of autonomy) [3]. In addition, connected vehicles communication (V2X) is facilitated through wireless technologies such as dedicated short range communications (DSRC) and long-term evolution-advanced (LTE-A) which allow CAVs to transmit relevant information to other vehicles, roadside infrastructure, and devices. The implementation of these technologies are expected to drastically reduce the approximately 94% of serious crashes in which human error plays a decisive role [4].

While there are many advantages associated with the implementation of CAV technologies, the potential dangers associated with them should not be neglected. The safe operation of CAVs is highly dependent on the information they obtain from their sensors and wireless communication sources. As a result, there exist a great number of sources in which anomalous/false information may be present, which poses great risk to the safe operation of CAVs. The anomalous/false information may be from a non-malicious source, i.e., faulty sensors or environmental hazard, or from a malicious source, i.e., a cyber-attack. It should be noted that both origins of anomalous/false information represent significant threats to the safe operation of CAVs. Detecting these anomalies is often challenging as they may be rather rare, leading to skewed training data for potential detection models. In addition, the rate of anomaly may vary over time, rendering any *a priori* detection criteria ineffective.

Much work has been done in the development of algorithms that can successfully detect such anomalies in CAVs (e.g., see surveys by [5], [6]). A significant number of these anomaly detection algorithms apply classification algorithms, where labeled data instances are used to train a model (classifier). For instance, in binary classification, the model is trained to provide a probability that the data are non-anomalous or anomalous. The resulting probability is then compared with a pre-defined threshold, e.g., 0.5, to classify the data into one of the two classes (non-anomalous versus anomalous).

Generally, the classification threshold is determined to balance sensitivity (the proportion of anomalous instances correctly classified) and specificity (the proportion of non-anomalous instances correctly classified); however, this can lead to over prioritizing a type of classification error. That is, based on the choice of the classification threshold, two types

of errors may occur: (1) classifying an anomalous instance as non-anomalous, i.e., false negatives; (2) classifying a non-anomalous instance as anomalous, i.e., false positives. False negatives are a clear concern in most applications. However, false positives can also have major consequences. For instance, in the classification of anomalous sensor values, false positives may eventually lead to the false exclusion of non-anomalous sensor data from the data fusion process. This can have a detrimental impact on the quality of fused data and potentially compromise the autonomous operation of the vehicle.

In this study, we develop a partially observable Markov decision process (POMDP) model to dynamically prescribe the classification threshold used in an anomaly detection algorithm, in real-time, to minimize the total expected ‘cost’ of false positives/negatives during a CAV trip. This extends the framework developed in [7]. Specifically, the POMDP model makes decisions about the classification threshold as a function of the belief about the core state of the system, the perceived past anomaly detection performance during a CAV trip, the classification threshold used during the past epoch, and an imperfect ‘observation’ received about the current state of the system. The inclusion of these parameters assures that both long-term and short-term past performance and beliefs about the state of the system are considered when making decisions.

Solving POMDP models using exact approaches is often intractable due to the curse of dimensionality [8]. Reinforcement learning (RL) based approaches can hence be used to solve such problems. In RL, for a system in state s , an agent chooses an action a to maximize its total discounted future rewards in a given environment [8]. This process is typically repeated until the most likely sample paths are all explored and the ‘best’ actions and rewards starting from any state are estimated. If the size of the policy space is too large, RL is typically not tractable as it requires too many iterations to explore the most likely sample paths to accurately estimate ‘best’ actions and rewards. Deep reinforcement learning (DRL) attempts to alleviate this problem by using the information collected from limited exploration of the policy space to estimate the actions and rewards starting from the states that are not often, if ever, explored.

In this study, we solve the POMDP model using the asynchronous advantage actor critic (A3C) algorithm [9] and compare the results with three benchmark models. Specifically, we use a convolutional neural network (CNN) and a CNN empowered Kalman filter (CNN-KF), as developed in [10], and a convolutional recurrent neural network (CRNN) as benchmarks. To facilitate comparisons, we keep the definitions for anomalies and the training, validation, and testing processes consistent with those used by [10]. We provide numerical experiments in which we compare the performance of the POMDP model with benchmark policies, i.e., multiple CNN and recurrent neural networks (RNN) based models with a fixed threshold throughout a trip. We provide numerical experiments for trips with various parameter settings to obtain further insights.

The main contributions of this work are as follows: (i) We embed CNN into a Bayesian framework, where instead of relying on the *raw* predictions of CNN on whether

sensors are faulty/attacked, we use the CNN predictions as ‘imperfect observations,’ fed into a Bayesian framework for further processing; (ii) we further incorporate the recent trip trajectory (or history) into the Bayesian framework to more actively react to environmental changes and anomaly rates during a trip. The developed framework is extremely flexible and allows for a large degree of control over penalizing false positives/negatives to various degrees in different applications; (iii) we use A3C to solve a mathematical POMDP model, as opposed to existing studies using A3C where the underlying model is a structured emulator (e.g., a video game).

The rest of the paper is organized as follows. In Section II, we provide a summary of the related works. In section III, we describe our framework. Specifically, we provide details for the POMDP model and the A3C training process and how the CNN is incorporated in the classification process. In Section IV, we describe the data set we use for training, validating, and testing. In Section V, we perform extensive numerical experiments. Lastly, we conclude in Section VI.

II. RELATED WORKS

A. Circumstances of Anomalous Event

There is a rapidly growing body of literature illustrating the potential dangers of CAV technologies resulting from anomalous/false information [11], [12], [13], [14]. For example, environmental conditions such as bright light or heavy snow may contribute as a non-malicious source of anomalous/false information as they may negatively affect CAV’s sensor readings [15]. Similarly, illustrating the injection of anomalous/false information with a malicious intent, researchers showed that it is possible to interfere with LiDAR systems of CAVs by artificially creating obstacles in the perceived environment using a laser and a pulse generator [16]. As a result, anomalous/false sensor values can result in dangerous circumstances for CAVs. It is therefore critical that anomaly detection techniques can detect anomalous/false information in CAVs.

B. Anomaly Detection Methods

Many anomaly detection methods have been considered in the literature. These include one-class methods such as support vector machines (SVM) [17], and density-based anomaly detection methods such as local outlier factor (LOF) [18]. The one class SVM relies only on “normal” data and do not require anomalies for training. The LOF produces an anomaly score that represents data points that are outliers. Although these methods are generally versatile in the types of anomalies they detect, they often fail to respond to potential changes in the rate of anomalies during a CAV trip.

Another prominent approach to anomaly detection in CAVs is (binary) classification [7], [10], [19]. Although these methods have shown great promise, they are often used in isolation and in a myopic manner. That is, after being built, a classification model remains unchanged regardless of the past success rate experienced within the system of interest [20]. Indeed, most models, even when they are designed to be used in an online fashion, such as throughout a CAV trip, they are simply trained to present the best performance

(e.g., high accuracy, sensitivity, or F-1 score) under an *expected trajectory*, e.g., 5% anomaly rate. Although such a strategy *on average* works well, it falls short of reacting to the real-time information experienced/gained throughout a specific *sample path or sequence of events*. Online machine learning methods, such as concept drift [21], try to combat this issue by (re-)learning the model parameters over time, in response to the new experiences gained.

C. Dynamic Thresholding in Anomaly Detection

To improve anomaly detection and address some of the existing issues, a decision-making system is needed to quickly respond to changes in the environment, without a long time lag or too much dependency on long-term memory. This can be achieved by dynamically adjusting the classification threshold, as opposed to a constant threshold, based on the complexity of the environment and previously learned information (e.g., performance metrics in the last m time epochs) to respond to the temporal changes in a timely manner. Dynamic thresholds have been proposed in various applications. Within the autonomous vehicle domain, adaptive thresholding has been successfully used in the past. For instance, a combination of generalized likelihood ratio method with adaptive threshold has been used to detect abrupt signal changes, outperforming a constant threshold [22]. In the broader context of anomaly detection, dynamic thresholding has been combined with genetic algorithms, von Mises-Fisher theory, and Long Short-Term Memory (LSTM) for anomaly detection in water distribution systems, and in nodes in web-based systems and spacecraft communications respectively, [23], [24], [25]. Although, dynamic thresholding has had context specific applications across select domains, to the best of our knowledge, the framework presented in this study is the first to address dynamic classification thresholding for anomaly detection in multi-channel datastreams in the transportation domain.

D. Reinforcement Learning in Anomaly Detection

Advances in computation capabilities in the past decade have expanded the use of DRL algorithms in many domains. The emergence of policy gradient-based methods such as the A3C [9], [26] algorithm have enabled the use of a continuous state space and action space. This development addressed the curse of dimensionality faced by Q-learning approaches [27] for large problems. Despite the potential of DRL techniques in anomaly detection, a review of the literature indicates that its application is very limited in this domain. Anomaly detection using DRL has been proposed in [28] to learn binocular fixations and in [29] for a threshold-free detector in network time series problems. However, these studies do not focus on anomaly detection in transportation and the models do not incorporate observations in a hierarchical manner, as proposed in our framework.

III. METHODS

In this section, we develop a framework to adaptively prescribe the anomaly classification threshold to improve the anomaly

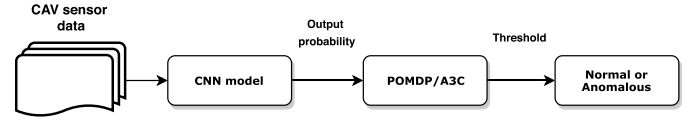


Fig. 1. Overview of the anomaly detection framework. At each time epoch, the sensor readings collected within the past several time epochs are used as an input to the CNN model. The CNN model outputs the probability of the sensor data being anomalous at each time epoch. Next, the CNN output probabilities, plus a few additional problem-specific features, are fed into the POMDP model as ‘imperfect observations.’ The POMDP model is solved using the A3C algorithm to obtain the policy that prescribes the anomaly classification threshold for any given system belief state, discriminating non-anomalous and anomalous sensor readings.

detection performance. In this study, we assume the true state of the system, i.e., non-anomalous or anomalous, is unknown. Hence, we use a Bayesian model to update the beliefs about the underlying state of the system. In this Bayesian model, specifically the POMDP model, we associate ‘costs’ to incurring false positives/negatives, and develop a solution strategy that aims to minimize the trip-specific total expected cost. Specifically, we solve this model with the A3C algorithm since POMDPs suffer from the curse of dimensionality and the problem investigated in this study has a large state space and continuous action space. As a result, exact methods are intractable for the problem investigated here.

In the following, we first provide an overview of the framework. Next, we discuss the two main components used in developing and solving the models, namely, POMDP and A3C algorithm. Fig. 1 illustrates an overview of the anomaly detection framework proposed in this study. At each time epoch, the sensor readings collected within the past several time epochs (using a fixed width sliding window, determined through preliminary experiments) are used as an input to the CNN model. It should be noted that the CNN model can be replaced by any other classification model producing a probability of belonging to a certain class. The CNN model outputs a probability of the sensor data being anomalous at each time epoch. Next, the CNN output probabilities, plus a few additional problem-specific features, are fed into the POMDP model. The POMDP model is solved using the A3C algorithm to obtain the stochastic policy that prescribes an anomaly classification threshold for any given system belief state. This policy can be used to discriminate between non-anomalous and anomalous readings by the control system of the CAV in real-time.

The CNN model used in this study is based on that developed in [10]. This CNN model uses a traditional image recognition architecture, and the Adam optimizer from Tensorflow in Python is implemented to minimize binary cross-entropy [30]. In addition, to reduce the risk of overfitting, random dropout and an early stopping techniques are used to monitor the validation set accuracy.

A. POMDP

A partially observable Markov decision process (POMDP) is described as a six-tuple (S, A, O, P, Ψ, R) with a set of states S , a set of actions A , a set of observations O , a transition

probability matrix P , an information matrix Ψ , and a reward function R , which is dependent upon the state and the action taken. At each decision epoch $t = 0, 1, 2, \dots, T : T < \infty$ the system is at a core state $s_t \in S$, which represents the true state of the system. We let $S = \{0, 1\}$, where the states ‘0’ and ‘1’ correspond to non-anomalous and anomalous states, respectively.

Note that the true system state is not fully observable. Hence, we let $\mathbf{q}_t = [q_0, q_1]$ denote the belief about the core state at time t , where q_0 and q_1 denote the probabilities that system is in non-anomalous and anomalous states, respectively. Note that $q_0 + q_1 = 1$. To more explicitly account for the immediate history, i.e., last m time epochs, we augment the belief vector $\mathbf{q}_t$, with the *perceived* past anomaly detection performance during a CAV trip, and the classification threshold used during the past epoch. Specifically, we let $b_t = [\mathbf{q}_t, \alpha, \beta, a_{t-1}] \in B$ denote the belief state of the process at time t , where α and β denote the weighted expected rates of false negatives (FN) and false positives (FP) over the past $m + 1$ decision epochs, respectively, and a_{t-1} denotes the action, i.e., the classification threshold, used in the previous time epoch $t - 1$.

Recall that the system core state is not fully known; it is only partially observable to the model. Hence, at each time epoch t , an observation $o_t \in O = [0, 1]$ is made, which provides imperfect information about the system core state. In our context, this observation is the CNN-estimated probability that the system is in the anomalous state. Note that the CNN is trained offline prior to its use within the POMDP. Once an observation is made, an action $a_t \in A = [0, 1]$ is taken, as a function of the belief state b_t and the observation o_t to adjust the the classification threshold.

At every decision epoch, the system incurs an immediate reward, which is a function of the action a_t taken in the belief state b_t . This immediate reward accounts for whether the threshold chosen is expected to correctly detect anomalous and non-anomalous readings. That is, the system is rewarded by a unit of one for true positives (TP) and true negatives (TN), and is penalized by a unit of one for false positives (FP) and false negatives (FN), i.e., $R_{TP} = R_{TN} = 1$ and $R_{FP} = R_{FN} = -1$. In addition, it is further rewarded/penalized based on the weighted expected rates of FN and FP over the past $m + 1$ decision epochs, α and β , without any full knowledge of the true system state/ground truth. The reward function $R_t(\cdot)$ for time period t depends on the belief, observation, and action, and is given as follows:

$$R_t(b, o, a) = \begin{cases} q_0 \cdot R_{TN} + q_1 \cdot R_{FN} + \alpha \cdot R_{FN}, & \text{if } o_t < a_t, \\ q_0 \cdot R_{FP} + q_1 \cdot R_{TP} + \beta \cdot R_{FP}, & \text{if } o_t \geq a_t. \end{cases}$$

In the beginning of time epoch $t + 1$, the belief state b_t is updated to obtain b_{t+1} . Specifically, the belief about the new core state $s' \in S$ at time $t + 1$ is updated as follows:

$$q_{s'} = \frac{\Psi(\mathbb{1}_{\{o \geq a\}}|s') \sum_{s \in S} P(s'|s) \cdot q_s}{\sum_{s' \in S} \Psi(\mathbb{1}_{\{o \geq a\}}|s') \sum_{s \in S} P(s'|s) \cdot q_s} \quad (1)$$

where $P(s'|s)$ denotes the probability that system core state transitions from state s to state s' , $\mathbb{1}_{\{\cdot\}}$ is an indicator function, and $\Psi(\mathbb{1}_{\{o \geq a\}}|s')$ denotes the probability that the observed system state is anomalous/non-anomalous, given the core state s' . Note that while $P(s'|s)$ is independent of a , the updated beliefs are dependent upon a . In addition, the weighted expected rates of FN and FP are respectively updated as follows:

$$\alpha' = \frac{\alpha \cdot m + \mathbb{1}_{\{o < a\}} \cdot q_1}{m + 1}, \quad (2)$$

$$\beta' = \frac{\beta \cdot m + \mathbb{1}_{\{o \geq a\}} \cdot q_0}{m + 1}. \quad (3)$$

Finally, we let γ denote the discount factor and $V_t(b)$ give the maximum total expected discounted reward starting from belief state b_t . Hence, for $t < T$, we have

$$V_t(b) = \max_{a \in A} \{R_t(b, o, a) + \gamma \sum_{o \in O} \sum_{s' \in S} \Psi(\mathbb{1}_{\{o \geq a\}}|s') \times \sum_{s \in S} q_s \cdot P(s'|s) \cdot V_{t+1}(b)\}. \quad (4)$$

For the terminal case $t = T$, i.e., end of the trip, a lump sum expected discounted reward-to-go is obtained, i.e., $V_T(b) = R_T(b, o, a)$.

B. A3C

We use the A3C algorithm to solve the developed POMDP model. In the A3C algorithm, being an *actor-critic* method, a critic learns the value function while the actor interacts with the environment attempting to learn a policy to maximize/minimize a given objective function. Among the common DRL algorithms such as deep Q-network (DQN), double DQN, etc. [9], [31], A3C generally achieves the best performance and it is suitable for both discrete and continuous spaces [9]. In addition, the training time for A3C is substantially less and, in general, obtains more robust policies compared to other DRL techniques. The A3C algorithm achieves this by exploring the environment with multiple agents in parallel on multiple threads. While A3C was selected due to its superior performance compared with other DRL methods, the solution to the proposed POMDP model can be obtained using alternative algorithms.

To reduce the correlation between training episodes, other DRL algorithms often utilize a technique called *experience replay*. A3C reduces correlation by creating a copy of the environment for each agent during the training process and updating the global/master network *asynchronously*. Therefore, after every gradient update, each of the individual agents synchronize their weights with the global network.

The A3C algorithm [9] maintains a policy $\pi_t(a|b; \phi)$, where a denotes the action and b denotes the belief at time period t , and ϕ denotes the parameters of the global actor network. The algorithm also estimates a value function $V_t(b; \phi_v)$, for belief, b , at time period t using parameters ϕ_v for the global critic network. The value function is updated with forward view returns. The policy and the value networks are updated after every episode of length t_{max} or when a terminal state is reached.

TABLE I
PARAMETERS USED IN POMDP MODEL & A3C

Hyperparameter	Value	Description
Maximum Episodes	100	Total number of training paths generated
Episode Epochs	200	Total number of epochs for each training path
Regularization, η	0.001	Encourages exploration of various
Update Iterations	10	Epochs between global parameter updates
Step Number, $\mathcal{W}$	10	Number of forward view time steps
Critic Learning Rate	0.001	RMSProp optimizer rate for critic network
Actor Learning Rate	0.001	RMSProp optimizer rate for actor network
NN Nodes	100	Number of nodes in each hidden layer
Window Size, m	10	Past epochs used to weight FN and FP
Discount Factor, γ	1	Parameter used to discount future rewards
Initialized α and β	0.0	Weighed expected rate over $m + 1$ epochs

The A3C algorithm uses *advantage*, as opposed to Q-values in many other DRL algorithms, to estimate how advantageous certain actions are for a particular state. The advantage function $\mathcal{A}_t(\cdot)$, starting from time t , is estimated as follows:

$$\mathcal{A}_t(b, a; \phi, \phi_v) = \sum_{i=0}^{\mathcal{W}-1} \gamma^i R_{t+i}(b, o, a) + \gamma^{\mathcal{W}} V_{t+\mathcal{W}}(b; \phi_v) - V_t(b; \phi_v), \quad (5)$$

where $R_t(\cdot)$ denotes the immediate reward for period t defined in Section III.A, and $\mathcal{W}$ denotes the number of forward view time steps and is bounded above by t_{max} .

The gradient update to minimize total loss can be seen as

$$\nabla_{\phi'} \log \pi_t(a|b; \phi') \mathcal{A}_t(b, a; \phi, \phi_v), \quad (6)$$

where ϕ' is the thread-specific network parameters. Also, [9] found that if the entropy of policy π is added to the objective function, exploration is encouraged which limits the premature convergence to sub-optimal deterministic policies. As a result, the gradient of the objective function which includes an entropy regularization term with respect to the policy parameters is given as

$$\nabla_{\phi'} \log \pi_t(a|b; \phi') \mathcal{A}_t(b, a; \phi, \phi_v) + \eta \nabla_{\phi'} H(\pi_t(b; \phi')), \quad (7)$$

where H is the policy entropy and the parameter η controls the strength of the entropy regularization term.

To solve the POMDP model with the A3C algorithm, we use TensorFlow [32] in Python where we utilize the RMSProp optimizer [33] for training the neural networks of the actor and critic. The trained actor-critic network is shallow consisting of a few dense layers. Once trained, this network dynamically alters the detection threshold of the CNN model enabling the online detection of anomalies. Table I illustrates a summary of the A3C hyperparameters as well as the POMDP parameters used in this study.

IV. DATA

The data used in the numerical experiments for this study are obtained from the research data exchange (RDE) database for the Safety Pilot Model Deployment (SPMD) program [34]. The SPMD program was conducted to demonstrate the feasibility of CAVs, investigating V2X technologies in real-world conditions. Data were collected from on-board vehicle devices as well as roadside units (RSUs) including

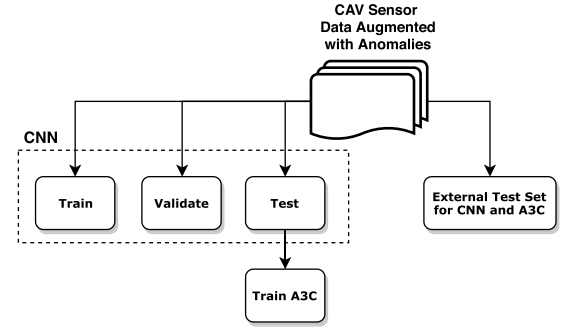


Fig. 2. Illustration of data breakdown and flow of information. The CNN model is trained and validated using CAV sensor data. The CNN model is then used to generate probability streams of the system being anomalous on an internal test set. These probabilities are used as observations in the POMDP model trained by the A3C algorithm. Finally, an external test set is used to compare the performance of the CNN-only and the POMDP model.

basic safety messages (BSM), signal phase and timing (SPaT) messages, vehicle trajectories, and data for driver-vehicle interactions. Data were collected for approximately 3,000 vehicles over a period of two years with a collection frequency of 100 ms for selected variables.

Consistent with [10], in-vehicle speed, GPS speed, and in-vehicle acceleration are extracted from the SPMD data set and used in this numerical study. This data set is collected based on real-world vehicle use. As such, it is important to note that although theoretically all these datastreams have known physical relationships, in practice, inconsistencies are often observed, e.g., speed measurements from in-vehicle sensors and GPS do not completely match. This can be due to a variety of reasons, such as the quality of the sensors, their degree of white noise, inherent delay in measurements (i.e., lack of time synchronization) across sensors [35], [36], etc. In this study, to introduce anomalies, we use the anomaly simulation process found in [10] to generate the ‘bias’ and ‘gradual drift’ anomaly types. Note that we define an anomaly as any artificial data inserted into the sensor datastream. As such, consistent with the literature [10], [37], anomalies are inserted to overlay non-anomalous sensor measurements, modeled from the SPMD data, at the specified rates ξ (dependent on the experiment). Specifically, for the bias anomaly type, the anomaly magnitude is sampled from a uniform distribution $\mathcal{U}(0, 5)$, where the sampled magnitude is added to all true sensor readings for a time period of 10 epochs. This anomaly results in a sudden increase to magnitude of one sensor’s data. Similarly, for the gradual drift anomaly type, a vector of linearly increasing values from 0 to 2, for a duration of 10 epochs are added to true sensor readings for a single sensor at a time. This results in a gradual change of sensor readings, which has been shown to be more difficult to identify [10].

Fig. 2 illustrates how the data set (consisting of in-vehicle speed, GPS speed, and in-vehicle acceleration) that is augmented with anomalous readings is used to train and test the various components of the framework. First, a portion of the data is set aside to train and validate the CNN model. The trained CNN model is then used to generate probabilities of the system being anomalous on an internal test set. These

probability streams are used in the POMDP model as *observations* when training the A3C algorithm. Finally, to evaluate the POMDP model's performance, an external/unseen test set is used and the results are presented in the numerical study.

V. RESULTS

In this section, we introduce benchmark methods and comparative metrics, and perform various analyses to investigate the anomaly detection performance of the models discussed in Section III. We first investigate the performance of the POMDP and benchmark models under single anomaly types, specifically, bias and drift, where we vary the anomaly rate. This experiment explores the potential of using the POMDP model in a dynamic road environment where the rate of anomaly changes as the CAV drives through the network. Next, we compare the performance of POMDP and benchmark models under a 'mixed' anomaly type, with bias and gradual drift anomalies being equally likely to occur, where we vary the anomaly rate. This experiment further helps showcase the robust nature of the proposed model under the anomaly type uncertainty in a real-world environment. Lastly, we provide an illustrative example of the POMDP model's dynamic threshold for an anomaly encountered in a sample path.

A. Benchmarks

In this section, we provide three state-of-the-art anomaly detection methods that are used as benchmarks in this study.

CNN: We use the CNN models consistent with the previous work [10]. Specifically, for each CNN model, a 3×10 sliding window is used on retrospectively collected data from sensors to produce 'images' for training and testing. When testing, if the CNN-estimated probabilities are above a pre-defined classification threshold ρ , the CNN classifies the image as anomalous, and as non-anomalous otherwise.

CNN-KF: The CNN-KF model is used to further improve upon the detection and identification performance of CNN. Following [10], the CNN-KF consists of two layers, i.e. CNN and Kalman filter (KF) with a χ^2 -detector. First, the CNN model processes the 'images' from the sensor data to identify whether an anomalous reading exists or not. The sensors with anomalous readings are then excluded if the CNN-estimated probabilities exceed the classification threshold ρ . Consequently, the readings from normal sensors are separately fed into adaptive KFs with failure detectors for further examination and anomaly detection. As for the KF, we assume a linear state-space model as follows:

$$\begin{aligned} x(k) &= \mathcal{T}x(k-1) + w(k-1) \\ z(k) &= \mathcal{M}x(k) + v(k), \end{aligned} \quad (8)$$

where $x(k) \in \mathbb{R}^m$ is the vector of state variable at time k , $z(k)$ is a vector of sensor measurements with measurement matrix $\mathcal{M} \in \mathbb{R}^{p \times m}$, $\mathcal{T} \in \mathbb{R}^{m \times m}$ is state-transition matrix, w and v are the process and measurement noises, respectively.

Adjunct with the KF, we use a χ^2 -detector to construct χ^2 test statistics, which determines whether the new measurement falls into the gate region with the probability determined by the gate threshold ϵ . Let $v(k)$ denote the innovation at

time k , which is the difference between the measurement $z(k)$ and the predicted measurement $\hat{z}(k|k-1)$, and $C(k)$ denote covariance matrix of innovation. Then the gate region is defined by

$$G_\epsilon = \{z : (z - \hat{z}(k|k-1))^T C(k)^{-1} (z - \hat{z}(k|k-1)) \leq \epsilon\}. \quad (9)$$

The χ^2 test statistics for each measurement is defined as

$$t(k) = v(k)^T C(k)^{-1} v(k). \quad (10)$$

Note that both $v(k)$ and $C(k)$ can be obtained as intermediate products of the KF.

CRNN: CRNN is a hybrid method which combines CNN and RNN. The CRNN applies CNN layers followed by RNN layers. In CNN layers, the middle level features are extracted and then RNN layers are applied to encode temporal dependencies. CRNN have been applied to sequential data of various types including, speech, audio signals, and video [38], [39]. The CRNN used in this paper starts with three 2D convolutional layers each followed by ReLU activation and max-pooling. After convolution layers, permute and reshape layers are added to prepare the output for input to the recurrent layer. Then, three LSTM layers are added followed by dropout with a rate of 10%. Finally, the RNN layers are followed by two fully connected layers with random dropout with a rate of 10% between the layers.

B. Metrics

Throughout our analyses, we evaluate the performance of the models in terms of accuracy, sensitivity, specificity, positive predictive value (PPV), and F1 score. We consider F1 score, a balance between sensitivity and PPV, as the main metric of comparison between the models. To highlight the percentage improvement in error rate between the F1 scores of the benchmarks and POMDP model, we introduce the metric $\Delta\text{Improve}$, i.e.,

$$\Delta\text{Improve} = \frac{\frac{\sum_{\xi \in \Xi} (F1_{\text{POMDP}} - F1_{\text{Benchmark}})}{|\Xi|}}{100 - \frac{\sum_{\xi \in \Xi} F1_{\text{Benchmark}}}{|\Xi|}} \cdot 100\% \quad (11)$$

where Ξ denotes the set of anomaly rate environments considered and $|\Xi|$ denotes its cardinality. For all experiments, we test the models using 10 distinct sample paths and present the mean and 95% confidence interval (CI) for the performance metrics.

C. Performance Under Single Anomaly Type

In this section, we investigate the effect of having different anomaly rates in a trip under a single anomaly type. First we conduct experiments to illustrate the impact of a mismatch between training and testing environments for a typical CNN-based anomaly detection benchmark at a high level. We then conduct thorough experiments where we compare the performance of the POMDP model with those of the three benchmark models trained using pre-determined anomaly rate environments, under different anomaly rate environments.

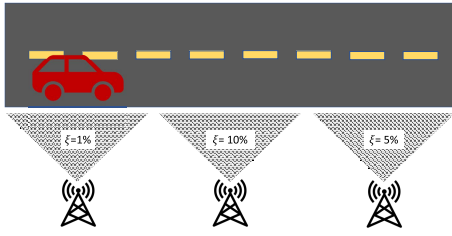


Fig. 3. Illustration of a sample path with different rates of anomaly for road segments with a fixed length. In this sample path, the CAV goes through three road segments where it communicates with RSUs that can inject anomalies at different rates, i.e., starting from $\xi = 1\%$, anomaly rate first increases to $\xi = 10\%$ and then decreases to $\xi = 5\%$.

TABLE II

THE MEAN AND 95% CI FOR F1 SCORE OF THE CNN MODEL, OBTAINED FOR VARIOUS TRAINING AND TESTING COMBINATIONS OF ANOMALY RATE UNDER BIAS ANOMALY TYPE

		Anomaly rate in testing		
		$\xi = 1\%$	$\xi = 5\%$	$\xi = 10\%$
Anomaly rate in training	$\xi = 1\%$	89.91 $\pm$ 2.00	88.51 $\pm$ 0.76	84.25 $\pm$ 0.43
	$\xi = 5\%$	87.60 $\pm$ 2.94	94.30 $\pm$ 0.61	94.24 $\pm$ 0.36
	$\xi = 10\%$	77.91 $\pm$ 5.13	91.86 $\pm$ 1.09	93.99 $\pm$ 0.45

First, we train and test a typical CNN-based model under various anomaly rates for the bias anomaly type. Specifically, we vary the anomaly rate to be $\xi \in \Xi = \{1\%, 5\%, 10\%\}$, for each of which we train a CNN-based model. Note, that these anomaly rates result in a skewed data set and are selected to reflect a potential real-world scenario in which anomalies are uncommon. The resulting models are then each applied to testing environments with three different anomaly rates, $\xi \in \Xi$. Therefore, this experiment explores the robustness of a typical CNN-model's detection capabilities within "unknown" environments. This is motivated by the real-world scenario in which the CAV does not know the anomaly rate of the road segment it is in, hence it may be unclear which trained model (with regard to anomaly rate) to use in these circumstances. Such a circumstance is illustrated in the experimental design in Fig. 3.

Table II illustrates the mean and 95% CI for the F1 score obtained from a typical CNN model with mismatched training and testing anomaly rates for the bias anomaly type. Overall, note that a model that is trained using a given anomaly rate, presents different performances when tested under different anomaly rates. For instance, when testing a typical CNN-based model that is trained on $\xi = 1\%$ (first row in Table II) on anomaly rates $\xi \in \Xi$, the resulting F1 score on average ranges between 84.25%–89.91%. Indeed in this case, the model that is trained and tested in the same type of environment, i.e., $\xi = 1\%$ for both training and test sets, presents the highest performance. As seen in Table II, such trend generally holds; given an anomaly rate in the training data set, better performance is achieved when using the same anomaly rate in the testing. The only exception observed in Table II is in the case of $\xi = 10\%$ in testing. In this case, the CNN model performs slightly better if it is trained on $\xi = 5\%$, rather than $\xi = 10\%$. Overall, these results demonstrate

TABLE III

THE MEAN AND 95% CI FOR F1 SCORE OF THE CNN MODEL, OBTAINED FOR VARIOUS TRAINING AND TESTING COMBINATIONS OF ANOMALY RATE UNDER GRADUAL DRIFT ANOMALY TYPE

		Anomaly rate in testing		
		$\xi = 1\%$	$\xi = 5\%$	$\xi = 10\%$
Anomaly rate in training	$\xi = 1\%$	90.74 $\pm$ 1.35	90.66 $\pm$ 0.63	88.61 $\pm$ 0.69
	$\xi = 5\%$	87.42 $\pm$ 1.99	93.55 $\pm$ 0.46	94.45 $\pm$ 0.27
	$\xi = 10\%$	80.49 $\pm$ 3.30	92.75 $\pm$ 0.66	94.66 $\pm$ 0.24

that when an environment's anomaly rate is unknown the resulting training/testing mismatch will negatively impact the performance of the anomaly detection model. Similar results are seen under the gradual drift anomaly type, presented in Table III. The drawback highlighted here motivates the use of dynamic thresholding to achieve the pre-trained algorithm's full potential.

Next, we compare the performance of the POMDP model with those of the three benchmark models (CNN, CNN-KF, CRNN) under different, pre-determined anomaly rate environments. In the following experiments we set ρ to the general best performing value under the selected anomaly type for each model. Table IV illustrates the anomaly detection performance of the models for the bias anomaly type, with benchmark models trained on the average anomaly rate of $\xi = 5\%$, under all test anomaly rate environments $\xi \in \Xi$. As seen in the table, the POMDP model outperforms the CRNN benchmark model in terms of F1 score, averaged across the three test environments (paired t -tests p-value < 0.05). There is no statistical difference between the POMDP model and CNN and CNN-KF benchmark models performance (paired t -tests p-value > 0.05). These differences are reflected in the large positive $\Delta\text{Improve}$ for POMDP versus CRNN, and the relatively small $\Delta\text{Improve}$ magnitude for POMDP versus CNN and CNN-KF.

Similarly, Table V illustrates the anomaly detection performance of the models for the gradual drift anomaly type, with benchmark models trained on the average anomaly rate of $\xi = 5\%$, under all test anomaly rate environments $\xi \in \Xi$. As seen in the table, the POMDP model underperforms the CNN benchmark model in terms of F1 score, averaged across the three test environments (paired t -test p-value < 0.05), but performs statistically similar to CNN-KF and CRNN benchmark models (paired t -test p-value > 0.05). This is reflected in a negative $\Delta\text{Improve}$ for POMDP versus CNN, and the very small $\Delta\text{Improve}$ magnitude for POMDP versus CRNN and CNN-KF.

Finally, to draw further insights, we investigate the impact of anomaly magnitude and duration in bias and drift anomaly types on the performance of the POMDP model. Our analysis shows that, as expected, the POMDP's performance improves in anomaly magnitude in both bias and drift anomaly types, which suggests that larger anomalies are generally easier to detect. Furthermore, the POMDP's performance decreases in anomaly duration in drift anomaly type. This is mainly because larger duration implies a slower pace in drift, which is generally more difficult to detect. Overall, these observations

TABLE IV

THE MEAN AND 95% CI ANOMALY DETECTION PERFORMANCE OF THE BIAS ANOMALY TYPE FOR ANOMALY RATE $\xi \in \{1\%, 5\%, 10\%\}$

	$\xi = 1\%$				$\xi = 5\%$				$\xi = 10\%$			
	POMDP	CNN	CNN-KF	CRNN	POMDP	CNN	CNN-KF	CRNN	POMDP	CNN	CNN-KF	CRNN
Accuracy	98.73 ± 0.26	98.63 ± 0.28	98.53 ± 0.33	98.50 ± 0.3	96.87 ± 0.35	96.91 ± 0.37	97.16 ± 0.36	96.62 ± 0.40	94.39 ± 0.32	94.55 ± 0.34	95.43 ± 0.34	94.43 ± 0.44
Sensitivity	91.05 ± 2.81	91.40 ± 2.00	91.88 ± 1.97	91.79 ± 1.80	90.80 ± 0.83	91.23 ± 0.85	92.44 ± 0.78	90.98 ± 0.80	89.26 ± 0.35	89.74 ± 0.34	91.73 ± 0.38	89.77 ± 0.47
Specificity	99.27 ± 0.25	99.14 ± 0.28	99.00 ± 0.32	98.97 ± 0.36	99.38 ± 0.26	99.27 ± 0.29	99.12 ± 0.31	98.97 ± 0.34	99.19 ± 0.31	99.05 ± 0.34	98.89 ± 0.39	98.79 ± 0.44
PPV	89.71 ± 3.52	88.20 ± 3.77	86.62 ± 4.14	86.31 ± 4.51	98.38 ± 0.69	98.08 ± 0.77	97.73 ± 0.81	97.32 ± 0.90	99.05 ± 0.36	98.89 ± 0.40	98.73 ± 0.44	98.60 ± 0.50
F1 score	90.31 ± 2.26	89.69 ± 2.39	89.09 ± 2.66	88.85 ± 2.72	94.43 ± 0.53	94.53 ± 0.56	95.00 ± 0.56	94.04 ± 0.62	93.90 ± 0.30	94.10 ± 0.32	95.10 ± 0.33	94.00 ± 0.43
$\Delta\text{Improve}$	POMDP vs. CNN: 1.25 ± 1.85				POMDP vs. CNN-KF: -3.56 ± 4.19				POMDP vs. CRNN: 7.09 ± 4.02			

TABLE V

THE MEAN AND 95% CI ANOMALY DETECTION PERFORMANCE OF THE GRADUAL DRIFT ANOMALY TYPE FOR ANOMALY RATE $\xi \in \{1\%, 5\%, 10\%\}$

	$\xi = 1\%$				$\xi = 5\%$				$\xi = 10\%$			
	POMDP	CNN	CNN-KF	CRNN	POMDP	CNN	CNN-KF	CRNN	POMDP	CNN	CNN-KF	CRNN
Accuracy	98.48 ± 0.31	98.60 ± 0.27	98.50 ± 0.30	98.51 ± 0.29	96.45 ± 0.21	96.47 ± 0.19	96.44 ± 0.21	96.41 ± 0.21	94.70 ± 0.33	94.62 ± 0.34	94.73 ± 0.29	94.48 ± 0.36
Sensitivity	89.48 ± 0.51	89.39 ± 0.56	89.44 ± 0.54	89.43 ± 0.50	90.17 ± 0.26	89.98 ± 0.25	90.14 ± 0.22	89.89 ± 0.33	90.40 ± 0.39	90.09 ± 0.46	90.43 ± 0.41	90.30 ± 0.45
Specificity	99.13 ± 0.30	99.26 ± 0.25	99.14 ± 0.30	99.16 ± 0.27	99.08 ± 0.28	99.19 ± 0.24	99.09 ± 0.28	99.15 ± 0.24	98.97 ± 0.34	99.12 ± 0.29	99.01 ± 0.31	98.63 ± 0.30
PPV	88.21 ± 3.52	89.78 ± 3.21	88.37 ± 3.43	88.58 ± 3.07	97.62 ± 0.70	97.90 ± 0.61	97.66 ± 0.68	97.88 ± 0.61	98.87 ± 0.38	99.03 ± 0.32	98.91 ± 0.33	98.50 ± 0.32
F1 score	88.77 ± 1.75	89.53 ± 1.55	88.84 ± 1.69	88.96 ± 1.62	93.74 ± 0.37	93.77 ± 0.29	93.75 ± 0.31	93.67 ± 0.39	94.45 ± 0.26	94.35 ± 0.26	94.48 ± 0.22	94.22 ± 0.27
$\Delta\text{Improve}$	POMDP vs. CNN: -2.90 ± 1.73				POMDP vs. CNN-KF: -0.32 ± 1.24				POMDP vs. CRNN: 0.92 ± 2.37			

TABLE VI

THE MEAN AND 95% CI ANOMALY DETECTION PERFORMANCE OF THE MIXED ANOMALY TYPE FOR ANOMALY RATE $\xi \in \{1\%, 5\%, 10\%\}$

	$\xi = 1\%$				$\xi = 5\%$				$\xi = 10\%$			
	POMDP	CNN	CNN-KF	CRNN	POMDP	CNN	CNN-KF	CRNN	POMDP	CNN	CNN-KF	CRNN
Accuracy	98.71 ± 0.15	98.62 ± 0.18	98.61 ± 0.19	98.14 ± 0.34	95.03 ± 0.37	95.89 ± 0.37	96.09 ± 0.37	95.86 ± 0.47	92.73 ± 0.42	92.97 ± 0.41	93.79 ± 0.37	93.50 ± 0.47
Sensitivity	88.45 ± 2.00	88.84 ± 1.98	89.27 ± 1.84	90.17 ± 1.28	87.14 ± 0.63	87.65 ± 0.64	88.45 ± 0.66	88.82 ± 0.79	86.12 ± 0.85	86.75 ± 0.83	88.43 ± 0.83	88.24 ± 0.79
Specificity	99.38 ± 0.24	99.26 ± 0.27	99.22 ± 0.28	98.66 ± 0.41	99.38 ± 0.18	99.28 ± 0.20	99.23 ± 0.21	98.75 ± 0.29	99.30 ± 0.26	99.16 ± 0.20	99.13 ± 0.29	98.75 ± 0.38
PPV	90.51 ± 3.13	88.94 ± 3.38	88.47 ± 3.35	81.90 ± 4.37	95.65 ± 0.48	98.04 ± 0.53	97.95 ± 0.53	96.71 ± 0.72	99.21 ± 0.28	99.05 ± 0.28	99.03 ± 0.31	98.61 ± 0.40
F1 score	89.36 ± 1.17	88.77 ± 1.32	88.76 ± 1.30	85.70 ± 2.19	92.39 ± 0.51	92.55 ± 0.54	92.96 ± 0.55	92.60 ± 0.72	92.20 ± 0.50	92.50 ± 0.47	93.43 ± 0.44	93.13 ± 0.50
$\Delta\text{Improve}$	POMDP vs. CNN: 12.11 ± 5.93				POMDP vs. CNN-KF: 7.40 ± 6.31				POMDP vs. CRNN: 18.94 ± 5.49			

are consistent with intuition and due to space limitations, the detailed results are not included.

D. Performance Under Mixed Anomaly Type

In this section we investigate the generalizability of the models to multiple types of anomalies. Specifically, anomalies can be randomly generated to be of types bias or gradual drift. Hence, this more closely represents real-world conditions in which the anomaly type *and* the anomaly rate are unknown *a priori*. In the new experiments, we again set ρ to the general best performing value under the mixed anomaly type for each model, and re-evaluate all performance metrics for CNN, CNN-KF, and CRNN, and compare the results with those of the POMDP model. As in Section V-C, we use the anomaly rates $\xi \in \Xi = \{1\%, 5\%, 10\%\}$.

Table VI illustrates the anomaly detection performance of the models for the mixed anomaly type, with benchmark models trained on the average anomaly rate of $\xi = 5\%$, under all test anomaly rate environments $\xi \in \Xi$. As seen in the table, the POMDP model outperforms all benchmark models in terms of F1 score, averaged across the three test environments (paired t -test p -value < 0.05). This is reflected in large positive $\Delta\text{Improve}$ for POMDP versus CNN, CNN-KF, and CRNN.

E. An Illustrative Example

In this section, we present an illustrative example to provide further insights on the observed difference in performance between the CNN and the POMDP models for a bias anomaly encountered in a sample path. The example shows how the POMDP model can improve upon the performance of the

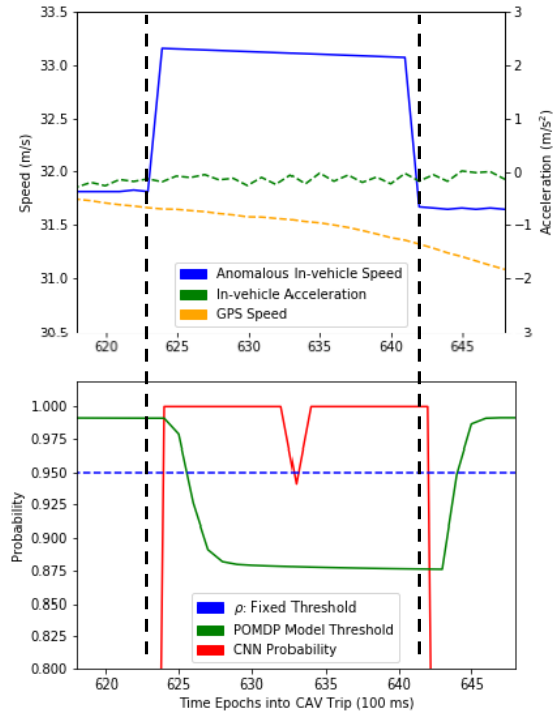


Fig. 4. Illustration of a bias anomaly encountered in a sample path. The GPS speed, In-vehicle speed augmented with an anomaly, the CNN output probability, and the threshold specified by A3C are illustrated.

CNN model by changing its threshold to detect anomalous instances missed by the CNN. Fig. 4 illustrates a bias anomaly encountered in a sample path. More specifically, the top panel of the figure presents anomaly-augmented in-vehicle speed as well as the non-anomalous in-vehicle acceleration and GPS speed. The bottom panel of the figure shows the real-time CNN output probability of an anomaly, the fixed CNN threshold of 0.95, as well as the POMDP adaptive threshold specified by A3C. The CNN model predicts the sensor data are anomalous if its output probability exceeds the fixed threshold value; otherwise the system is classified as non-anomalous. Similarly, for the POMDP model, the sensor data are anomalous if the CNN-estimated probability is greater than the POMDP model threshold, and they are marked as non-anomalous, otherwise.

As seen in the top panel, the anomaly starts at time epoch 623. As seen in the bottom panel, both the CNN and the POMDP model take at least one time epoch to recognize that an anomaly has occurred, and hence fail to correctly classify the first instance of the anomaly. However, immediately after, the CNN output probability sharply rises (to 1.0), putting it above the fixed threshold of 0.95, hence detecting the anomaly. This is mainly because the bias anomaly type is relatively simplistic and hence, easy to detect for the CNN. Similarly, the POMDP model, which was previously using a relatively large classification threshold based on the confidence of the CNN, now starts to lower its threshold. This leads to the CNN probability exceeding the now modified POMDP threshold at epoch 625, correctly marking the anomaly.

At epoch 633, the CNN model outputs a probability below 0.95, meaning that the CNN incorrectly classifies the data as

non-anomalous, i.e., a FN is incurred. However, for POMDP, its lowered threshold ensures that the CNN output probability remains greater than its current threshold resulting in the POMDP model, continuing to correctly classify the anomaly. Note that the fall in CNN probability occurs as the anomaly passes 10 epochs and due to the CNN window-size it re-normalizes the in-vehicle speed. Both models correctly detect the remainder of the anomaly. After the last instance of the anomaly occurs at time epoch 643, the CNN probability lowers to approximately 0.00 again, and the POMDP threshold is raised to approximately 0.99 again with both models correctly classifying the data as non-anomalous in the following time epochs.

VI. CONCLUSION AND FUTURE WORK

The goal of this study is to develop a framework to detect anomalous behaviors in CAVs. Specifically, we develop an approach that embeds a CNN classification model into a Bayesian framework with the main objective of improving the performance of the primary classification model. The Bayesian framework consists of a POMDP model which is solved using the A3C algorithm. To more readily react to environmental changes and anomaly rates during a trip, we also account for recent trip information in this framework. The framework, which is expected to be generalizable to many changing environments (as the model can be retrained to detect various types of anomalies), is flexible in that it allows for the pairing of any classification model with the POMDP model. The framework is also generally adaptable to various sensor types and redundancy (allowing it to be applied to various contexts including multiple anomalous datastreams), and inherently accounts for sensor uncertainty, in particular time delay. In addition, it provides control over the cost of false negatives/positives, dependent on the specific modeling purposes.

Our results show that the addition of the POMDP model generally leads to statistically improved performance in F1 score, especially under the more challenging case of mixed anomaly types. This is mainly because, in contrast to the other methods, the POMDP model is adaptive in nature, and it benefits from the inclusion of the partial information that it infers from the ‘observations.’

It is interesting to note that the performance of the CNN model can be varied by changing the fixed classification threshold, e.g., lowering the threshold would improve sensitivity, at the expense of specificity. However, unlike the dynamic thresholding approach developed in this study, this approach neither accounts for the environmental changes experienced during the trip, nor allows for adapting to the history of the sample path.

This research contributes to the field of ITS safety. Specifically, the proposed framework addresses the issue of using a single trained classification model to detect anomalies. In real-world conditions, the anomaly rate of road segments are not known in advance. Hence, as illustrated in this study, using a hierarchical decision model may improve the anomaly detection capabilities of the primary classification model.

The study is subject to certain limitations. In our experiments, similar to previous studies in the literature, the

anomalous instances in the speed sensor data are simulated. Therefore, although this is consistent with the recent literature and is done due to the paucity of real-world anomalous CAV data, it imposes implicit assumptions about the availability of anomalous data for training and the similarity of anomalous data types in training and testing. To partially combat these limitations, in our experiments, we use two anomaly types, namely, bias and gradual drift. However, additional testing is required for other anomaly types and sensor types using simulations, as well as training and testing using real-world anomalous data to more accurately measure the degree of generalizability of the models to novel anomaly types. This also includes training and testing using intelligent, simultaneous, malicious attacks on multiple sensors. It is envisioned that data collected from recently completed and ongoing pilot studies may contribute to the availability of real-world data including anomalous instances.

In future work, it may be useful to test additional models as the primary classification model in the framework. Alternatively, new architectures with a combination of neural networks where one of which outputs a threshold for the other one may be explored. In addition, the POMDP model and calibration can be further extended to improve the model performance in both finite and infinite horizon applications. Also, other DRL techniques can be used to solve the POMDP model. Furthermore, implementing online learning with an annealing process can be used to allow for improved learning over a small training set. In addition, it may be interesting to use the stream of belief probabilities generated by the POMDP in real-time as an input to an additional decision making model to more accurately approximate the true state of the system.

REFERENCES

- [1] D. Ni, *Traffic Flow Theory: Characteristics, Experimental Methods, and Numerical Techniques*. Oxford, U.K.: Butterworth-Heinemann, 2015.
- [2] J. Rios-Torres and A. A. Malikopoulos, "A survey on the coordination of connected and automated vehicles at intersections and merging at highway on-ramps," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 5, pp. 1066–1077, May 2017.
- [3] NHTSA. *Preliminary Statement of Policy Concerning Automated Vehicles*. Accessed: Jun. 12, 2019. [Online]. Available: <https://www.nhtsa.gov/technology-innovation/automated-vehicles-safety>
- [4] S. Singh, "Critical reasons for crashes investigated in the national motor vehicle crash causation survey," Nat. Highway Traffic Safety Admin., Washington, DC, USA, Tech. Rep. DOT-HS-812-115, 2015.
- [5] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, p. 15, Jul. 2009, doi: 10.1145/1541880.1541882.
- [6] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, Jan. 2016.
- [7] Z. Liu, A. Khojandi, X. Li, R. L. Davis, and R. Kamaleswaran, "A machine learning-enabled partially observable Markov decision process framework for early sepsis prediction," *INFORMS J. Comput.*, pp. 1–19, 2022. [Online]. Available: <https://pubsonline.informs.org/doi/abs/10.1287/hjoc.2022.1176>
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [9] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [10] F. van Wyk, Y. Wang, A. Khojandi, and N. Masoud, "Real-time sensor anomaly detection and identification in automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 1264–1276, Apr. 2019.
- [11] J. Petit and S. E. Shladover, "Potential cyberattacks on automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 546–556, Apr. 2015.
- [12] K. Koscher *et al.*, "Experimental security analysis of a modern automobile," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 447–462.
- [13] A. Weimerskirch and R. Gaynier, "An overview of automotive cybersecurity: Challenges and solution approaches," in *Proc. TrustED CCS*, 2015, p. 53.
- [14] S. Parkinson, P. Ward, K. Wilson, and J. Miller, "Cyber threats facing autonomous and connected vehicles: Future challenges," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 11, pp. 2898–2915, Nov. 2017.
- [15] J. Moavenzadeh and N. Lang, "Reshaping urban mobility with autonomous vehicles lessons from the city of Boston," World Economic Forum, 2018. [Online]. Available: https://www3.weforum.org/docs/WEF_Reshaping_Urban_Mobility_with_Autonomous_Vehicles_2018.pdf
- [16] M. Harris, "Researcher hacks self-driving car sensors \$60 lidar spoofing device generates fake cars, pedestrians and walls," IEEE Spectrum, New York, NY, USA, 2015. Accessed: Aug. 23, 2022. [Online]. Available: <https://spectrum.ieee.org/researcher-hacks-selfdriving-car-sensors>
- [17] B. Schölkopf, R. C. Williamson, A. J. Smola, J. Shawe-Taylor, and J. C. Platt, "Support vector method for novelty detection," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 12, 1999, pp. 582–588.
- [18] M. Breunig, H. P. Kriegel, R. T. Ng, and J. Sanders, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.
- [19] R. A. A. Habeeb, F. Nasaruddin, A. Gani, I. A. T. Hashem, E. Ahmed, and M. Imran, "Real-time big data processing for anomaly detection: A survey," *Int. J. Inf. Manage.*, vol. 45, pp. 289–307, Apr. 2019.
- [20] C. Kruegel, D. Mutz, W. Robertson, and F. Valeur, "Bayesian event classification for intrusion detection," in *Proc. 19th Annu. Comput. Secur. Appl. Conf.*, 2003, pp. 14–23.
- [21] J. Gama, I. Žliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surveys*, vol. 46, no. 4, pp. 1–37, Apr. 2014.
- [22] A. Kalmuk, O. Granichin, O. Granichina, and M. Ding, "A dynamic threshold based algorithm for change detection in autonomous systems," *IFAC-PapersOnLine*, vol. 49, no. 13, pp. 141–145, 2016.
- [23] J. Arad, M. Housh, L. Perelman, and A. Ostfeld, "A dynamic thresholds scheme for contaminant event detection in water distribution systems," *Water Res.*, vol. 47, no. 5, pp. 1899–1908, Apr. 2013.
- [24] T. Idé and H. Kashima, "Eigenspace-based anomaly detection in computer systems," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2004, pp. 440–449.
- [25] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and non-parametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 387–395.
- [26] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [27] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [28] F. de La Bourdonnaye, C. Teuliere, T. Chateau, and J. Triesch, "Learning of binocular fixations using anomaly detection with deep reinforcement learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 760–767.
- [29] C. Huang, Y. Wu, Y. Zuo, K. Pei, and G. Min, "Towards experienced anomaly detector through reinforcement learning," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 8087–8088.
- [30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2014, pp. 1–15.
- [31] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1, pp. 2094–2100.
- [32] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Syst. Design Implement.*, 2016, pp. 265–283.
- [33] T. Tieleman and G. Hinton, "Lecture 6.5-RmsProp: Divide the gradient by a running average of its recent magnitude," *COURSERA, Neural Netw. Mach. Learn.*, vol. 4, no. 2, pp. 26–31, 2012.
- [34] D. Bezzina and J. Sayer, "Safety pilot model deployment: Test conductor team report," Intell. Transp. Syst. Joint Program Office U.S. Dept. Transp., Washington, DC, USA, Tech. Rep. DOT-HS-812, 2015, p. 171.
- [35] M. Lentmaier, B. Krach, and P. Robertson, "Bayesian time delay estimation of GNSS signals in dynamic multipath environments," *Int. J. Navigat. Observ.*, vol. 2008, pp. 1–11, Apr. 2008.
- [36] S. Björklund and L. Ljung, "A review of time-delay estimation techniques," in *Proc. 42nd IEEE Int. Conf. Decis. Control*, 2003, pp. 2502–2507.
- [37] F. Guo *et al.*, "Detecting vehicle anomaly in the edge via sensor consistency and frequency characteristic," *IEEE Trans. Veh. Technol.*, vol. 68, no. 6, pp. 5618–5628, Jun. 2019.

- [38] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Convolutional recurrent neural networks for music classification," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Mar. 2017, pp. 2392–2396.
- [39] Z. Zuo *et al.*, "Convolutional recurrent neural networks: Learning spatial dependencies for image representation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2015, pp. 18–26.



Jeremy Watts received the B.S. degree in physics and the M.S. degree in nuclear engineering. He is currently a Graduate Student Research Assistant with the Department of Industrial and Systems Engineering, The University of Tennessee Knoxville. His research interests include decision making and the application of machine learning in crosscutting fields.



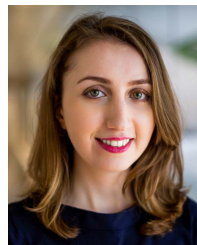
Franco van Wyk received the M.Eng. degree in engineering management from Stellenbosch University, South Africa, and the Ph.D. degree in industrial and systems engineering from The University of Tennessee Knoxville. He is currently a Senior Developer at CompariSure, Stellenbosch University. His research interests include optimal sequential decision making under uncertainty and safety in connected and automated vehicle systems.



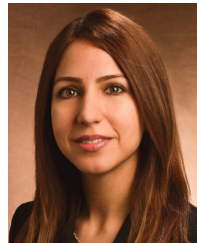
Shahrbanoo Rezaei received the B.S. and M.S. degrees in industrial engineering. She is currently a Graduate Student Research Assistant with the Department of Industrial and Systems Engineering, The University of Tennessee Knoxville. Her research interests include sequential decision making and machine learning.



Yiyang Wang received the M.S. degree in electrical engineering and computer science from the University of Michigan, Ann Arbor. He is currently pursuing the Ph.D. degree with the Department of Civil and Environmental Engineering. His research interests include reinforcement learning and cybersecurity in connected and autonomous vehicle technologies.



Neda Masoud received the B.S. degree in industrial engineering, the M.S. degree in physics, and the Ph.D. degree in civil and environmental engineering from the University of California at Irvine. She is currently an Assistant Professor with the Department of Civil and Environmental Engineering, University of Michigan, Ann Arbor. Her research interests include large scale optimization and machine learning with applications in shared-use mobility services and connected and automated vehicle systems. She is a member of INFORMS and TRB.



Anahita Khojandi (Member, IEEE) received the Ph.D. degree in industrial engineering from the University of Pittsburgh. She is currently an Associate Professor with the Department of Industrial and Systems Engineering, The University of Tennessee Knoxville. Her research interests include decision making under uncertainty, data mining, and machine learning with applications in a wide array of systems. She is a member of INFORMS, IISE, and SMDM.